

CRT Phase 2 : POST TEST

Name : Dewanshu Chirkhe

CRT Batch : B1

Branch : Data Science

Q1. [4M] Predict the output. What will print to the console when this Spring Boot application starts? Write the output in order and give a one-line reason for each.

```
@Component
public class MyBean {
    public MyBean() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("Init called");
    }
}
```

Output :

Constructor called
Init called

Reason:

Constructor called — Spring first creates the bean by calling its constructor.

Init called — After the bean is created and all dependencies are injected, Spring calls the `@PostConstruct` method.

Q2. [3M] The endpoint below compiles and runs but always returns an empty response body. Identify the mistake and write the corrected line only.

```
@Controller
@RequestMapping("/api/users")
public class UserController {
    @Autowired UserService service;

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        User u = service.findById(id);
        return new ResponseEntity<>(u, HttpStatus.OK);
    }
}
```

Answer:

Mistake: The class is annotated with `@Controller` instead of `@RestController`. With `@Controller`, Spring expects a view (HTML page), so the response body is empty.

Corrected line:

```
@RestController
```

Q4. [3M] What annotation should be added to the transfer() method below, and why? Write the annotation and explain in 2 lines what happens if it is missing.

```
@Service
public class BankService {
    @Autowired AccountRepository repo;

    public void transfer(Long fromId, Long toId, double amount) {
        Account from = repo.findById(fromId).orElseThrow();
        Account to = repo.findById(toId).orElseThrow();
        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);
        repo.save(from);
        repo.save(to);
    }
}
```

Answer:

Annotation: @Transactional

Why: This method performs two database saves. @Transactional ensures both saves succeed together or both are rolled back if anything fails, keeping data consistent.

If missing: If an error occurs after saving 'from' but before saving 'to', money is deducted from the sender but never credited to the receiver — causing data corruption.

Q6. [5M] Look at the JWT filter below. (a) Explain in 2 lines what this filter does. (b) There is one bug — the token extracted from the header will always be invalid. Find it and write the corrected line.

```
String header = req.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) {
    chain.doFilter(req, res);
    return;
}
String token = header.substring(6); // extract token
String username = jwtUtil.extractUsername(token);
if (username != null) {
    UsernamePasswordAuthenticationToken auth =
        new UsernamePasswordAuthenticationToken(username, null, null);
    SecurityContextHolder.getContext().setAuthentication(auth);
}
chain.doFilter(req, res);
```

Answer:

(a) What this filter does:

It reads the Authorization header from each incoming request. If the header contains a valid Bearer token, it extracts the JWT, gets the username from it, and sets the user as authenticated in Spring Security's context so the request can proceed.

(b) Bug:

header.substring(6) is wrong. "Bearer " has 7 characters (including the space). Using 6 skips only "Bearer" and includes the space as part of the token, making it always invalid.

Corrected line:

```
String token = header.substring(7); // "Bearer " is 7 characters
```

Q8. [4M] Explain the N+1 query problem in JPA in your own words with a simple example. What is the most common cause? Write one line of code that fixes it in a `@Query` method using JOIN FETCH.

Answer:

N+1 Problem:

When you load a list of N records, JPA fires 1 query to get the list, then N more queries to fetch related data for each record — totalling N+1 queries instead of 1.

Example:

Load 10 Orders → 1 query. Access `o.getItems()` for each Order inside a loop → 10 more queries.
Total = 11 queries.

Most common cause:

Using `@OneToMany` or `@ManyToOne` with LAZY loading and then accessing the lazy field inside a loop.

Fix using JOIN FETCH:

```
@Query("SELECT o FROM Order o JOIN FETCH o.items")
List<Order> findAllWithItems();
```

Q11. [3M] A developer runs the following code to load 10 orders and print their items. The console shows 11 SQL queries instead of 1. What is this problem called? Write the line of code that causes it and the one-line fix.

```
List<Order> orders = orderRepo.findAll();
for (Order o : orders) {
    System.out.println(o.getItems().size());
}
```

Answer:

Problem: N+1 Query Problem.

Line that causes it:

```
System.out.println(o.getItems().size());
```

This triggers a separate SQL query for each Order to fetch its items.

One-line fix:

```
@Query("SELECT o FROM Order o JOIN FETCH o.items")
```

Use this query in the repository instead of `findAll()`, so items are loaded in a single JOIN query.

Q13. [5M] What is a Circuit Breaker in microservices? Why is it needed? Write a Spring Boot method that calls an external payment-service using `RestTemplate` and applies a `Resilience4j @CircuitBreaker` with a fallback method that returns a default `PaymentResponse` object.

Answer:

What is a Circuit Breaker:

A pattern that stops calling a failing external service repeatedly. If a service fails too many times, the circuit 'opens' and requests are blocked temporarily. A fallback response is returned instead of crashing the caller.

Why needed:

Without it, if payment-service is down, your app keeps sending requests, wastes resources, and can cause a cascade failure bringing down other services too.

Spring Boot method with @CircuitBreaker:

```
@Service
public class PaymentService {

    @Autowired
    private RestTemplate restTemplate;

    @CircuitBreaker(name = "paymentService", fallbackMethod =
"fallbackPayment")
    public PaymentResponse makePayment(PaymentRequest req) {
        return restTemplate.postForObject(
            "http://payment-service/pay", req, PaymentResponse.class);
    }

    public PaymentResponse fallbackPayment(PaymentRequest req, Exception e) {
        return new PaymentResponse("FAILED", "Payment service unavailable. Try
later.");
    }
}
```

Q14. [6M] What is an API Gateway and what are 2 benefits of using one? Write a Spring Cloud Gateway application.yml that routes requests starting with /api/orders/** to order-service and /api/users/** to user-service. Use lb:// for load-balanced URIs.

Answer:

What is an API Gateway:

A single entry point for all client requests. It receives requests and routes them to the correct microservice, so the client only needs to know one URL.

2 Benefits:

1. Clients talk to a single URL — no need to know individual service addresses.
2. Centralises cross-cutting concerns like authentication, rate limiting, and logging.

application.yml:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/api/orders/**
        - id: user-service
          uri: lb://user-service
```

```
predicates:  
  - Path=/api/users/**
```

Q15. [4M] What is Service Discovery in microservices and why is it needed instead of hardcoding IP addresses? Name the tool you used in class. Write a RestTemplate bean configuration that enables it to resolve service names automatically.

Answer:

What is Service Discovery:

A mechanism where microservices register themselves with a central registry by name. Other services look up the registry to find their address instead of using hardcoded IPs.

Why needed:

IP addresses of services change when they restart or scale. Hardcoding IPs breaks the app. Service Discovery always gives the current, correct address.

Tool used in class: Eureka (Netflix Eureka Server)

RestTemplate bean (resolves service names via @LoadBalanced):

```
@Bean  
@LoadBalanced  
public RestTemplate restTemplate() {  
    return new RestTemplate();  
}
```

Q16. [6M] You have a monolithic Spring Boot app with modules: UserModule, OrderModule, ProductModule. Your team wants to split it into microservices. (a) List 3 benefits of this split. (b) List 2 challenges. (c) Briefly explain how Eureka and an API Gateway work together so a React frontend can talk to all three services through a single URL.

Answer:

(a) 3 Benefits:

1. Independent deployment and scaling — each service can be deployed or scaled without affecting others.
2. Fault isolation — if OrderModule crashes, UserModule and ProductModule stay up.
3. Team independence — separate teams can own and develop each service in parallel.

(b) 2 Challenges:

1. Inter-service communication adds complexity — network calls can fail, adding latency.
2. Distributed data management — each service has its own database, making cross-service queries harder.

(c) How Eureka and API Gateway work together:

Every microservice (user-service, order-service, product-service) registers itself with Eureka on startup. The API Gateway is the single entry point for the React frontend. When a request arrives (e.g., /api/orders/**), the Gateway queries Eureka to find the current address of order-service, then forwards the request to it. The React app only ever calls one URL — the Gateway — and never needs to know individual service addresses.

Q18. [4M] What is the difference between `@Mock` (Mockito) and `@MockBean` (Spring Boot Test)? When would you use each? Write one example of a test class declaration that uses `@MockBean` correctly with `@SpringBootTest`.

Answer:

@Mock (Mockito):

Creates a plain mock object without starting the Spring context. Faster. Used for pure unit tests where you manually inject dependencies.

@MockBean (Spring Boot Test):

Creates a mock AND registers it as a bean in the Spring application context. Used when testing a Spring component that needs its dependencies wired by Spring.

When to use:

Use `@Mock` when you don't need Spring at all (simple unit test).

Use `@MockBean` when you load the Spring context (`@SpringBootTest`) and want to replace a real bean with a mock.

Example:

```
@SpringBootTest
public class UserServiceTest {

    @MockBean
    private UserRepository userRepository;

    @Autowired
    private UserService userService;

    @Test
    void testFindUser() {
        when(userRepository.findById(1L)).thenReturn(Optional.of(new User(1L,
            "Dewanshu")));
        User result = userService.findById(1L);
        assertEquals("Dewanshu", result.getName());
    }
}
```

Q22. [5M] The React component below has a bug that causes it to fetch data in an infinite loop. (a) Identify exactly what is wrong. (b) Fix it. (c) Add a simple loading state that shows 'Loading...' while data is being fetched.

```
function UserList() {
    const [users, setUsers] = useState([]);

    useEffect(() => {
        fetch('/api/users')
            .then(res => res.json())
            .then(data => setUsers(data));
    }); // <-- notice anything?

    return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

Answer:

(a) What is wrong:

The `useEffect` has no dependency array. This means it runs after every render. When `setUsers(data)` is called, it triggers a re-render, which triggers `useEffect` again — causing an infinite loop.

(b) Fix:

Add an empty dependency array `[]` as the second argument to `useEffect`. This tells React to run the effect only once, on the first render (mount).

(c) Fixed component with loading state:

```
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => {
        setUsers(data);
        setLoading(false);
      });
  }, []); // empty array - runs only once on mount

  if (loading) return <p>Loading...</p>;

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```